



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Reading Grammars in Language Syntax

```
struct group_info init_groups = { .usage = ATOMIC_INIT(2) };

struct group_info *groups_alloc(int gidsetsize){
    struct group_info *group_info;
    int nblocks;
    int i;

    nblocks = (gidsetsize + NGROUPS_PER_BLOCK - 1) / NGROUPS_PER_BLOCK;
    /* Make sure we always allocate at least one indirect block pointer */
    nblocks = nblocks ? : 1;
    group_info = kmalloc(sizeof(*group_info) + nblocks*sizeof(gid_t *), GFP_USER);
    if (!group_info)
        return NULL;

    group_info->ngroups = gidsetsize;
    group_info->nblocks = nblocks;
    atomic_set(&group_info->usage, 1);

    if (gidsetsize <= NGROUPS_SMALL)
        group_info->blocks[0] = group_info->small_block;
```

Learning Outcome:

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations. Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Ask your partner:

- How can we verify the legality of a program?
- How are programs' statements made?

Can Do Statements:

- I can give information to understand how to read grammars in language syntax.
- I can explain different features related to language syntax and its grammars.
- I can identify detailed information about grammars in language syntax.



Let's Explore!

Reading Grammars in Language Syntax is a piece of cake



Instructions: Read the text below.

- Sergio:** Hi, Cristian, I would like us to practice the topic of language syntax, you know, we cannot fail that exam.
- Christian:** Sure! Let's review the topic. So far I remember, the syntax of a language defines whether or not the language is legal and if we can execute it successfully.
- Sergio:** yeah, I also remember that the language syntax has nothing to do with the functions of the program, but with the grammar.
- Christian:** Do you understand the concept of grammar?
- Sergio:** Yes, a grammar is a set of rules that define how to make statements, which are like sentences; those statements are made of tokens that are groups of characters equivalent to words in human languages.
- Christian:** You're right, then we can use the Extended Backus Form to define grammar; This notation differentiates terminal symbols or single tokens, from nonterminal symbols, that means a sequence of tokens. We define nonterminal symbols using grammar rules that show us the expanded sequence of detailed tokens. To read the grammar we start from left to right always with a nonterminal symbol.
- Sergio:** We can use the EBNF to verify if a statement is legal, right?
- Christian:** You can also use it to generate statements.
- Sergio:** Ok, anything else we are missing?
- Christian:** Yeah, the EBNF defines context-free grammars that always generate the same token fragments in the non-terminal variables. There are other techniques to read context-sensitive grammars that depend on the context to run.
- Sergio:** I want to add that in order to avoid ambiguity in our sentences, we have to add as many conditions as necessary because changing the syntax of the program can be a very complex process.
- Christian:** I think we are ready for the exam.



Let's Practice!

Now, Sergio can give some information about how to read grammars.

*Instructions: Complete the sentences by writing its corresponding word. Use capital letter where necessary. Use: **we can continue, how I can read, We can find, we can start with.***

1. Oh right! I want to know _____ grammars.
2. I know that to read a grammar, _____ any nonterminal symbol.
3. Then, _____ reading the grammar rule from left to right.
4. _____ a grammar rule for each nonterminal symbol encountered.

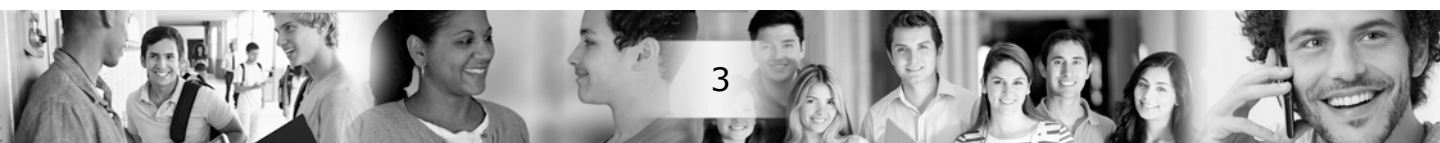


Let's Practice!

Sergio wants to practice some features related to language syntax.

Instructions: Select the correct answer.

1. What kind of symbol does the grammar in language syntax start with?
 - a. A terminal symbol.
 - b. A nonterminal symbol.
 - c. A legal symbol.
2. How do we read the underlining grammar rule?
 - a. From left to right.
 - b. From right to left.
 - c. From top to bottom.
3. Nonterminal symbols are made of sequences of...
 - a. symbols
 - b. steps
 - c. tokens
4. What's the equivalent of words in the programming language syntax?
 - a. Statements
 - b. Tokens
 - c. EBNF
5. The classification of techniques to read grammars in language syntax depends on...
 - a. How expensive they are
 - b. How exclusive they are
 - c. How expressive they are
6. People use syntax grammar to generate statements and...
 - a. correct them
 - b. verify them
 - c. delete them





Extension Activity

Let's read grammar in language syntax!

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search more information about how to read grammar in language syntax.
- Write ideas about grammar language syntax rules.
- Use the grammar covered in this unit to write your ideas.
- Discuss your ideas with your partners.

Self-Assessment

Yes

No

- I look for more information on grammar in language syntax.
- I explain what grammar in language syntax is.
- I share my ideas about grammar in language syntax with my classmates.

☐
☐
☐
☐
☐
☐


Target Words

- **detailed** (adjective): a complete description of something
e.g. This notation differentiates terminal symbols or single tokens, from nonterminal symbols, that means a sequence of tokens.
- **successfully** (adverb): to achieve something satisfactorily.
e.g. The syntax of a language defines whether or not the language is legal, and we can execute it successfully.
- **fragment** (noun): a piece or part of whole
e.g. The EBNF defines context-free grammars that always generate the same token fragments in the nonterminal variables.
- **sequence** (noun): a series of related facts or things
e.g. We can use the Extended Backus Form to define grammar. This notation differentiates terminal symbols, single tokens, from nonterminal symbols, sequence of tokens.
- **fail** (verb): in this case, to reprove
e.g. I would like to practice the topic of language syntax, you know, we cannot fail that exam.
- **expanded** (adjective): enlarged, widened.
e.g. We define nonterminal symbols using grammar rules that show us the expanded sequence of detailed tokens.



Learning Points

Verbs with Infinitive

There are some common verbs in English that introduce another action with the preposition **to** and the infinitive form of the verb.

When you introduce an action with verbs such as, want, would like, and other verbs to talk about likes and dislikes; you have to use the **infinitive** form of the verb preceded by the preposition **to**.

For example:

*We would like **to buy** a new laptop.*

*I want **to study** computer systems engineering.*

Verbs followed by the infinitives
Like to Want to Hate to Love to Decide to Attempt to Have to

Examples

- *I would like to practice the topic of language syntax.*
- *I want to add that in order to avoid ambiguity in our sentences, we have to add as many conditions as necessary.*



Learning Points

Modal Can for Possibility

You use **can** to talk about possibility or ability.

You use the **modal verb can** when you want to express possibility. A modal verb has special characteristics:

1. It has no conjugation, it's the same for all subject pronouns.
2. A modal verb is never introduced by a preposition, they usually come after or before subjects.
3. Modal verbs don't need auxiliary verbs to make interrogative or negative statements.

Examples:

*I **can** read the grammar of the program.*

*Tom **can** read the grammar too.*

*We **cannot** understand the topic.*

***Can** you make statements using tokens?*

Modal verbs act as their own auxiliaries when creating negative or interrogative statements.

Examples

- *We cannot fail that exam.*
- *The syntax of a language defines whether or not the language is legal and we can execute it successfully.*
- *We can use the EBNF to verify if a statement is legal, right?*